Provably Good Scheduling of LLM Agents under KV-Cache Pressure

[Author names] [1]

Abstract

LLM agents alternate between model calls and tool calls of unpredictable length, and many agents share one GPU. When KV-cache memory is scarce, the serving engine's choice of *which agent to serve next* decides how long every task takes, yet today's engines schedule individual requests first-come-first-served and agent-aware systems use heuristics without guarantees. We model agent serving as stochastic scheduling of task graphs with random tool delays on a batched engine with bounded KV memory, and derive a *task index* Z' that ranks whole agent tasks by their total cost to the GPU, including the memory they occupy and the recomputation that tool waits cause. We prove that scheduling by this index is within a constant factor of the optimal non-anticipatory policy (factor 2, tight, without memory limits; about 5–7 with KV memory at our measured A40 parameters); two independent verification passes checked every proof. We implemented the index on vLLM as a drop-in scheduler plug-in and client layer. On four A40 GPUs serving Qwen3-8B to multi-hop question-answering agents under KV pressure, it cuts the **median task completion time by 37%** (42 s to 27 s) and the mean by 19% relative to vLLM, improves **every one of seven workloads**, and beats a Continuum-style agent scheduler by 47% on the median.

1. Problem

An LLM agent task is a small program: it calls the model, calls tools (search, code, APIs) whose latency is long and random compared with a model step, often several in parallel (Kim et al., 2024), and calls the model again on the results (Yao et al., 2023). A serving engine such as vLLM (Kwon et al., 2023) sees only the individual model calls. It admits them first-come-first-served (FCFS), and while an agent waits for a tool, its KV cache either occupies scarce GPU memory or is evicted and later recomputed.

Why scheduling matters here. Under memory pressure only a few agents fit on the GPU at once, so the order in which tasks are served dominates latency. Per-request FCFS is a poor order for agents: a task's latency is set by its *last* model call, and a long task that arrives first blocks many short tasks behind it, each of which also holds memory across its tool waits.

Why it is new. Agent-aware systems schedule at the program level, with heuristics: Continuum (Li et al., 2025) pins an agent's KV cache for a time-to-live during tool calls and serves programs FCFS, Autellix (Luo et al., 2025) ranks programs by least attained service, and LAMPS (Shahout et al., 2024) ranks requests by memory consumption over time and chooses whether to keep or discard memory during API calls. None comes with a performance guarantee. On the theory side, recent online algorithms for KV-constrained LLM scheduling (Jaillet et al., 2025) treat single requests, with no tool calls, task graphs or random durations, and classical stochastic scheduling (Möhring et al., 1999; Skutella & Uetz, 2005) has no batching, KV memory or tool delays. To our knowledge, ours is the first scheduling policy for agent serving with a provable approximation guarantee.

Challenges. (1) Tool delays and output lengths are random and unknown when decisions are made. (2) GPU time and KV memory are coupled: evicting a waiting agent frees memory but costs recomputation, and a batched step slows down as more sequences join it. (3) Latency belongs to whole tasks, while the engine schedules individual calls. (4) A guarantee must survive contact with a real engine: vLLM's admission, preemption and caching rules.

2. Design

Model. A task J arrives at time r_J with weight w_J and is a directed acyclic graph of LLM nodes; tool calls are random delays on the edges. Node y has a new prompt of q_y tokens, a context of ρ_y tokens carried from earlier calls, and at most X_y output tokens. The engine runs batched steps of duration $a+cB+\alpha P$ for B decoding sequences and P prefill tokens, and the KV cache holds at most M tokens. On our A40

[1]Harvard University, CS 243 (Fall 2026). Midterm report. Correspondence to: [Author] <[email]>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

with Qwen3-8B we measured $a = 27.8\,\text{ms}$, $c = 0.54\,\text{ms}$ and $\alpha = 0.11\,\text{ms}$ per token. The objective is the expected weighted sum of task completion times, $\mathbb{E}[\sum_J w_J C_J]$.

The task index. Each task gets one number when it arrives,

$$Z'_J = \sum_{y \in J} \left[\alpha(q_y + \rho_y) + cX_y + \alpha(\ell_y + L_{\max}) + \frac{aX_y \ell_y}{M'} \right], \tag{1}$$

where $\ell_y = \rho_y + q_y + X_y$ is the node's peak KV footprint, $L_{\max}$ the largest footprint and M' the usable memory. The four terms are the node's prefill work (including recomputing evicted context after a tool wait), its decode work, a charge for the work it may displace when it is admitted, and its share of the memory-time area. Tasks are served in increasing order of Z'_J/w_J, a memory-aware generalization of Smith's rule (Smith, 1956). The index is fixed for the task's lifetime, so a task never loses its place to newer arrivals of the same size.

Guarantees. Let OPT be the best non-anticipatory policy (it knows distributions, not the future).

Theorem 1 (no memory limit)**.** *For task graphs with known node sizes and arbitrary random tool delays, on m parallel servers, the online preemptive policy that runs tasks in order of w_J/W_J (total work W_J) satisfies $\mathbb{E}[\sum w_J C_J] \le 2\,\text{OPT}$, and the factor 2 is tight.*

Theorem 2 (batched engine with KV memory)**.** *In the engine model above, the policy that ranks tasks by w_J/Z'_J, admits the longest priority prefix of requests whose reservations fit in memory, and evicts KV during tool waits, satisfies $\mathbb{E}[\sum w_J C_J] \le c \cdot \text{OPT}$ with $3\times$ engine speed and $M + L_{\max}$ memory, where c depends only on the ratio of recomputation to useful work; $c \approx 5$–7 at our measured parameters for M between 32k and 128k tokens.*

Theorem 1 extends Megow and Schulz (Megow & Schulz, 2004) to task graphs with random tool delays: delays enter only through the critical path. Theorem 2 is new; its key step charges every preemption to the arrival that caused it. We also proved a 2-approximation for chain tasks with random sizes (a Gittins-index variant), that keeping KV across tool waits never breaks Theorem 2 if kept KV is evicted first, and that the best keep/evict time-to-live for a known delay distribution is within $e/(e-1)$ of clairvoyant. Two independent verification passes checked every proof; each inequality is also checked exactly on small instances, and the algebra with the Z3 solver (7,772 automated tests).

Implementation (Figure 1). The system needs no fork of vLLM. A client layer (130 lines) computes Z'_J from token-count estimates when a task arrives and sends it as the priority of each of the task's model calls. A scheduler subclass (360 lines), loaded through vLLM 0.32's `--scheduler-cls` option, adds the engine rules the theory uses: priority-prefix admission with preemption, a single prefill per step, a low-priority class for speculative prefills, a soft time-to-live for KV kept during tool waits, and protection of cached prefixes that queued requests need. All rules are switches, so baselines run through the same code. In this report *Ours* is the index on top of vLLM's priority scheduler; the engine rules are evaluated as ablations (Section 4).

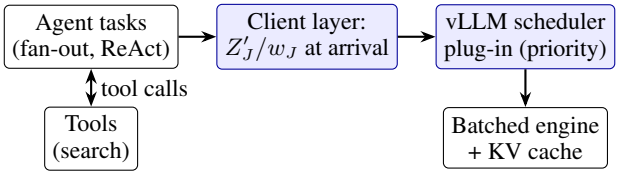


Figure 1. Implementation. The client layer computes each task's index once and attaches it as the priority of every model call the task makes; a scheduler plug-in loaded with `--scheduler-cls` applies it inside vLLM. Shaded boxes are ours.

Table 1. All tasks at 16k tokens of KV memory, seven seeds (653 paired tasks). "Faster": share of tasks that finish earlier than under vLLM.

Scheduler	Mean (s)	Median (s)	Faster
vLLM (FCFS)	50.3	42.0	–
Continuum-like	52.1	50.7	48%
Autellix-like*	65.9	47.3	49%
Ours (Z' index)	**40.6**	**26.7**	**79%**

*Run with our preemptive admission rule.

3. Initial Evaluation

Setup. Four NVIDIA A40 GPUs (48 GB), one vLLM 0.32 server per GPU, Qwen3-8B (Qwen Team, 2025) in bf16. Agents answer multi-hop HotpotQA questions (Yang et al., 2018) with a search tool over a pool of HotpotQA paragraphs, in two shapes: *fan-out* (plan, then 2, 4 or 8 parallel searches, then answer) and *ReAct* (2, 4 or 6 search-and-reason turns). Search results are up to 1,000 tokens; tool delays are uniform on 0.5–4 s with probability 0.7 and on 4–12 s otherwise. Tasks arrive as a Poisson process at 1 per second for 90 s (about 93 tasks per run). To emulate the memory pressure of a busier multi-tenant server, we cap the KV cache at 16k tokens (full capacity is 167k). Each *seed* fixes the arrival times, task mix and tool delays, and every scheduler sees exactly the same seed, so comparisons are paired task by task. We report task completion time, from arrival to final answer.

Baselines. *vLLM*: stock scheduler, requests FCFS. *Continuum-like*: our reimplementation of Continuum's policy (Li et al., 2025): KV pinned during tool calls with a time-to-live, programs served FCFS, pinned programs first. *Autellix-like*: Autellix's program-level least-attained-service ranking (Luo et al., 2025).

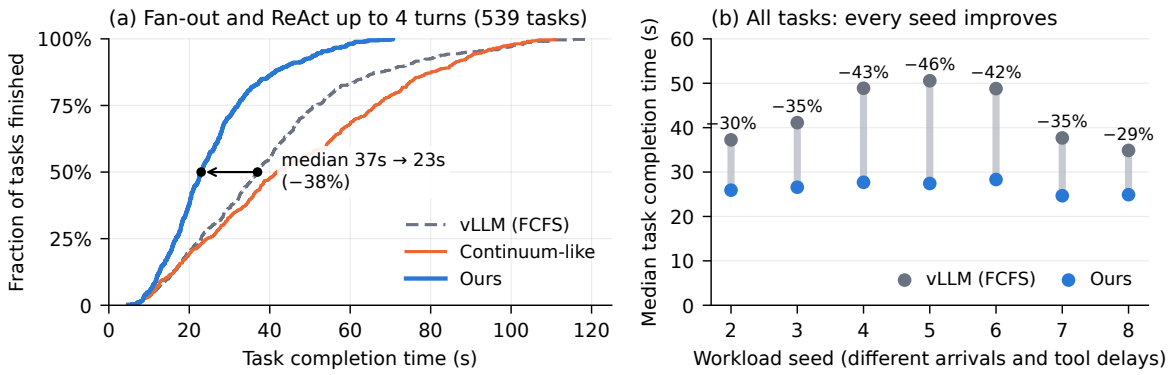


Figure 2. Task completion times at 16k tokens of KV memory, seven workloads. (a) Fan-out tasks and ReAct tasks with up to four turns (539 of 653 tasks): our index finishes 90% of them within 45 s versus 74 s with vLLM, and the slowest within 71 s versus 120 s. The longest tasks (six-turn ReAct, 17%) are discussed in Section 4. (b) Median over all tasks, per workload: our index improves every seed, by 29–46% (mean: 11–24%).

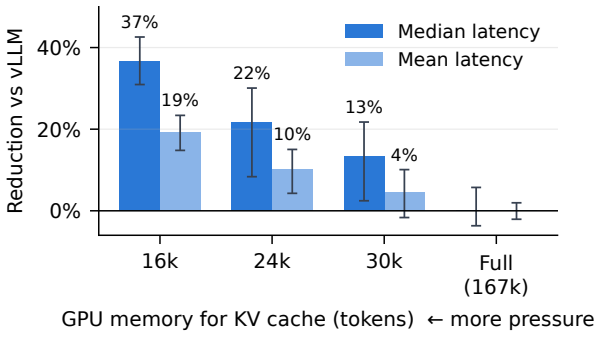


Figure 3. Latency reduction of our index over vLLM as KV memory grows (95% CIs). The tighter the memory, the larger the gain; with ample memory scheduling order barely matters and the index costs nothing.

Results. Figure 2 and Table 1 show the main result. Under memory pressure, scheduling by the task index cuts the median completion time from 42 s to 27 s (−37%, 95% CI 31–43%) and the mean from 50.3 s to 40.6 s (−19%, CI 15–23%), and 79% of tasks finish earlier. The improvement holds on every one of the seven workloads (Figure 2b). Against the Continuum-like scheduler, the median drops by 47% (CI 36–54%) and the mean by 22%. Confidence intervals come from a two-level bootstrap that resamples whole workloads and then tasks, so they include run-to-run variation.

The gain is broad across task types. For fan-out tasks the mean falls by 34% (32.9 s to 21.6 s), and for two-search fan-outs and two-turn ReAct agents by 41% and 45%. Fan-out tasks finish twice as fast as under the Continuum-like scheduler (21.6 s versus 42.8 s on average).

When does it help? Figure 3 repeats the experiment with more KV memory (three seeds each at 24k, 30k and full memory). The benefit grows with memory pressure: the median falls by 22% at 24k and 13% at 30k, and at full memory the index matches vLLM. This matches the model: the index matters when memory limits how many agents run at once, which is the regime of busy multi-tenant deployments, long contexts and large tool outputs.

4. Remaining Challenges and Plan

1. Isolate what the index contributes. With equal weights, Z' ranks tasks by estimated cost. We will compare it on equal footing with simpler rankings: smallest estimated task first, Autellix's ranking on stock admission (our Autellix-like run above used our preemptive admission), and a LAMPS-style memory-area ranking (Shahout et al., 2024). This decides whether the memory and recomputation terms of Eq. (1) matter in practice.

2. Protect the longest tasks with aging. Ranking by cost makes the largest tasks wait: six-turn ReAct agents (17% of tasks) finish 14% later than under vLLM, and their 90th percentile does not improve (106 s versus 101 s). Each of their seven model calls queues at the lowest priority, so the delay compounds. Our fix gives task J the priority $r_J + \kappa Z'_J/w_J$ (smaller is served first): its arrival time plus κ times its index. This interpolates between task-level FCFS ($\kappa = 0$) and the index ($\kappa \to \infty$), and it bounds overtaking: a task K that arrives after J is served first only if it arrives less than $\kappa(Z'_J/w_J - Z'_K/w_K) \le \kappa Z'_J/w_J$ seconds later, so no task waits behind an endless stream of smaller ones. The priority is still fixed at arrival, which Theorem 2's proof needs. The rule is implemented in the client layer and unit-tested; at 16k memory Z' ranges from 3.1 s (two-search fan-out) to 16.7 s (six-turn ReAct), so κ between 1 and 8 protects a long task from tasks arriving up to 2 minutes after

it. We will pick κ on the testbed and extend Theorem 2 to this priority using the overtaking bound.

3. Engine changes: make preemption and KV retention cheap. Theorem 2 needs admission in strict priority order with preemption. In our ablations this rule costs latency on long tasks, because vLLM preempts a request by discarding its KV cache and recomputing the whole context later; likewise, keeping KV across tool waits gave no measurable gain on top of the index at this scale. The scheduler plug-in already has the rules the proof uses (Section 2), but not the following changes to vLLM's memory management, which target these two costs. (a) *Preempt by swapping, not recomputing:* route preemptions and tool-wait evictions through vLLM's KV-offload connector, which copies blocks to CPU memory, so resuming costs a PCIe transfer instead of a prefill of the full context. Our model treats a swap as a fixed delay, like a tool call, so we expect the analysis to carry over. (b) *Evict by value, not recency:* vLLM evicts cached blocks least-recently-used. We will order them by the recomputation they save per unit of memory and expected remaining wait, using the tool-delay distribution the client attaches to each request; today we only move blocks whose time-to-live has expired to the front. Theorem 2 allows any eviction order among kept blocks, so this cannot break the guarantee. (c) *A price for memory:* our keep-or-evict analysis for many agents sharing memory is a threshold rule with a memory price; we will estimate that price online from the free-memory fraction, so agents keep KV across tool waits only when memory is plentiful. We will also look for an admission rule that needs no preemption and keeps a constant-factor guarantee, and test KV retention with longer contexts and tool outputs, where recomputation is expensive.

4. Broaden the evaluation. More arrival rates and memory sizes, a larger model, and measuring how close each run comes to the theory's lower bound on OPT, which tests whether the proven constant is pessimistic.

Timeline. Items 1 and 2 by the presentation (12/1); items 3 and 4 and the final report by 12/16. Our earlier study of KV preloading of tool results, which motivated the engine rules, is complete and serves as a fallback.

References

Jaillet, P., Jiang, J., Mellou, K., Molinaro, M., Podimata, C., and Zhou, Z. Online scheduling for LLM inference with KV cache constraints. *arXiv preprint arXiv:2502.07115*, 2025.

Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M. W., Keutzer, K., and Gholami, A. An LLM compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Li, H., Mang, Q., He, R., Zhang, Q., Mao, H., Chen, X., Cheung, A., Gonzalez, J., and Stoica, I. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. *arXiv preprint arXiv:2511.02230*, 2025.

Luo, M., Shi, X., Cai, C., Zhang, T., Wong, J., Wang, Y., Wang, C., Huang, Y., Chen, Z., Gonzalez, J. E., and Stoica, I. Autellix: An efficient serving engine for LLM agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.

Megow, N. and Schulz, A. S. On-line scheduling to minimize average completion time revisited. *Operations Research Letters*, 32(5):485–490, 2004.

Möhring, R. H., Schulz, A. S., and Uetz, M. Approximation in stochastic scheduling: The power of LP-based priority policies. *Journal of the ACM*, 46(6):924–942, 1999.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Shahout, R., Liang, C., Xin, S., Lao, Q., Cui, Y., Yu, M., and Mitzenmacher, M. Fast inference for augmented large language models. *arXiv preprint arXiv:2410.18248*, 2024.

Skutella, M. and Uetz, M. Stochastic machine scheduling with precedence constraints. *SIAM Journal on Computing*, 34(4):788–802, 2005.

Smith, W. E. Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3(1–2):59–66, 1956.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.